유한체 구현 분석 (Finite Fields Implementation Analysis)

개요

Plonky3는 다양한 유한체(finite field) 구현을 제공하여 서로 다른 성능 특성과 수학적 요구사항을 만족합니다. 각 유한체는 특정한 수학적 특성을 가지며, SIMD 최적화와 Montgomery 산술을 통해 고성능을 달성합니다. 이 문서에서는 Mersenne31, BabyBear, KoalaBear, Goldilocks 필드의 구현과 최적화 기법을 상세히 분석합니다.

유한체 구현 개요

지원하는 유한체들

| 필드 | 소수 § | 비트 크기 | Two-adicity | 특징 |

|------|----------|----------|----------|------|

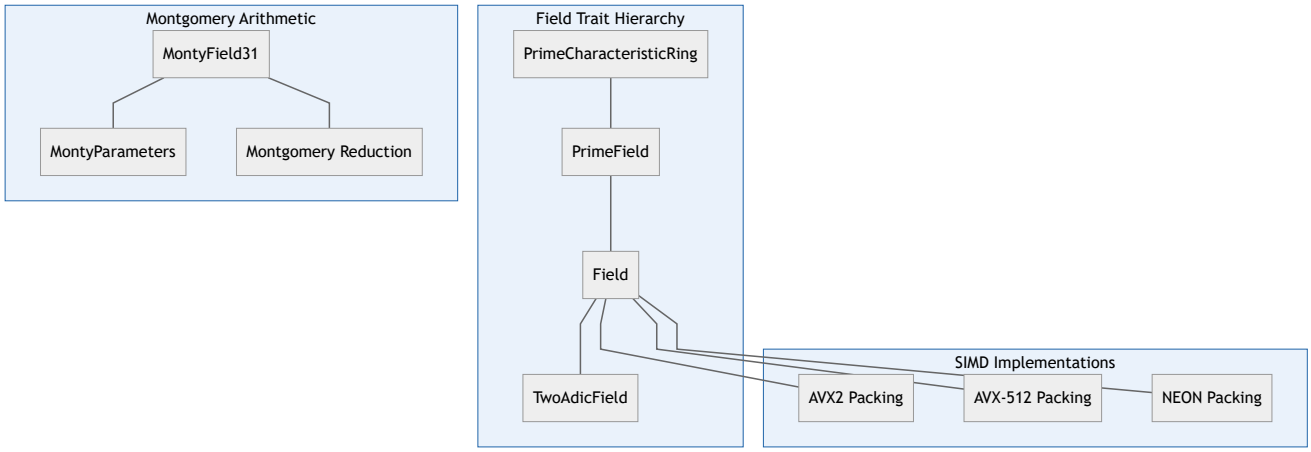
| Mersenne31 | $2^{31} - 1$ | 31-bit | 1 | Mersenne 소수, 빠른 모듈러 연산 |

| BabyBear | $2^{31} - 2^{27} + 1$ | 31-bit | 27 | 높은 two-adicity, FFT 최적화 |

| KoalaBear | $2^{31} - 2^{24} + 1$ | 31-bit | 24 | 3차 자기동형사상 지원 |

| Goldilocks | $2^{64} - 2^{32} + 1$ | 64-bit | 32 | 64비트 필드, 최고 성능 |

공통 아키텍처



Mersenne31 필드 (2^{31} - 1)

수학적 특성

Mersenne31은 $p = 2^{31} - 1 = 2,147,483,647$ 을 소수로 하는 유한체입니다.

핵심 특성:

- **Mersenne 소수**: 2의 거듭제곱에서 1을 뺀 형태의 소수
- **31비트 표현**: 32비트 정수 내에서 효율적 연산 가능
- **낮은 Two-adicity**: 2-adicity가 1로 FFT에는 제한적
- **빠른 모듈러 연산**: 특수한 형태로 인한 최적화 가능

구현 특징

```
const P: u32 = (1 << 31) - 1; // 2^31 - 1
```

```
pub struct Mersenne31 {
/// 31비트 내의 값, 정규화되지 않을 수 있음
pub(crate) value: u32,
}
```

핵심 최적화:

1. **회전 기반 2의 거듭제곱 연산**:

```
fn mul_2exp_u64(&self, exp: u64) -> Self {
// Mersenne 필드에서 2^k 곱셈은 k비트 좌회전
let exp = exp % 31;
let left = (self.value << exp) & ((1 << 31) - 1);
let right = self.value >> (31 - exp);
let rotated = left | right;
Self::new(rotated)
}
```

2. **효율적인 덧셈**:

```
fn add(self, rhs: Self) -> Self {
let (sum_i32, over) = (self.value as i32).overflowing_add(rhs.value as i32);
let sum_u32 = sum_i32 as u32;
let sum_corr = sum_u32.wrapping_sub(Self::ORDER_U32);
```

```
Self::new(if  over { sum_corr } else { sum_u32 })

}
```

3. **곱셈과 축약**:

```
fn  mul(self, rhs: Self) -> Self {

let  prod = u64::from(self.value) * u64::from(rhs.value);

from_u62(prod) // 62비트에서 31비트로 축약

}
```

성능 특성

- **장점**: 매우 빠른 기본 연산, 간단한 구현
- **단점**: FFT에 제한적 (낮은 two-adicity)
- **사용 사례**: 기본적인 다항식 연산, 해시 함수

BabyBear 필드 ($2^{31} - 2^{27} + 1$)

수학적 특성

BabyBear는 $p = 2^{31} - 2^{27} + 1 = 2,013,265,921$ 을 소수로 하는 유한체입니다.

핵심 특성:

- **높은 Two-adicity**: 2-adicity가 27로 FFT에 최적화
- **31비트 소수**: 32비트 정수 내에서 효율적 처리
- **Montgomery 형식**: 빠른 곱셈을 위한 Montgomery 산술 사용
- **확장 필드 지원**: 4차, 5차, 8차 확장 필드 구현

Montgomery 산술 구현

BabyBear는 `p3-monty-31` 크레이트의 `MontyField31` 구조를 사용합니다:

```
pub  type BabyBear = MontyField31<BabyBearParameters>;


impl MontyParameters for BabyBearParameters {
const PRIME: u32 = 0x78000001; // 2^31 - 2^27 + 1
const MONTY_BITS: u32 = 32;
const MONTY_MU: u32 = 0x88000001; // Montgomery 매개변수
}
```

Montgomery 축약 과정:

1. 곱셈 결과를 64비트로 확장
2. Montgomery 축약으로 32비트로 변환
3. 필요시 조건부 뺄셈으로 정규화

Two-adic 구조

```
impl TwoAdicData for BabyBearParameters {
const TWO_ADICITY: usize = 27;
// 2-adic 생성원들 (2^0부터 2^27까지)
const TWO_ADIC_GENERATORS: Self::ArrayLike = &BabyBear::new_array([
0x1, 0x78000000, 0x67055c21, 0x5ee99486, // ...
]);
```

```
}
```

확장 필드 구현

BabyBear는 여러 차수의 확장 필드를 지원합니다:

4차 확장 필드 ($\mathrm{BabyBear}^4$):

```
impl BinomialExtensionData<4> for BabyBearParameters {
const W: BabyBear = BabyBear::new(11); // 기약 다항식의 계수
const DTH_ROOT: BabyBear = BabyBear::new(1728404513);
const EXT_GENERATOR: [BabyBear; 4] = BabyBear::new_array([8, 1, 0, 0]);
}
```

KoalaBear 필드 ($2^{31} - 2^{24} + 1$)

수학적 특성

KoalaBear는 $p = 2^{31} - 2^{24} + 1 = 2{,}130{,}706{,}433$ 을 소수로 하는 유한체입니다.

핵심 특성:

- **3차 자기동형사상**: 곱셈군에서 $x \mapsto x^3$이 자기동형사상
- **Two-adicity 24**: FFT에 적합한 높은 2-adicity
- **특수 최적화**: 3차 거듭제곱근 계산에 최적화

3차 거듭제곱근 최적화

```
impl RelativelyPrimePower<3> for KoalaBearParameters {
```

```
/// KoalaBear에서 a^{1/3} = a^{1420470955}
fn exp_root_d<R: PrimeCharacteristicRing>(val: R) -> R {
exp_1420470955(val)
}
}
```

확장 필드에서의 곱셈 최적화

```
impl BinomialExtensionData<4> for KoalaBearParameters {
const W: KoalaBear = KoalaBear::new(3);
#[inline(always)]
fn mul_w<A: Algebra<MontyField31<Self>>>(a: A) -> A {
a.double() + a  // 3 * a = 2 * a + a (덧셈으로 최적화)
}
}
```

Goldilocks 필드 ($2^{64} - 2^{32} + 1$)

수학적 특성

Goldilocks는 $p = 2^{64} - 2^{32} + 1$ 을 소수로 하는 64비트 유한체입니다.

핵심 특성:

- **64비트 소수**: 가장 큰 필드 크기
- **최고 Two-adicity**: 2-adicity가 32

- **특수 구조**: $2^{96} = -1 \bmod p$ 의 특성 활용
- **고성능**: 64비트 아키텍처에서 최적 성능

특수 산술 최적화

2의 거듭제곱 곱셈:

```
fn mul_2exp_u64(&self, exp: u64) -> Self {
// 2^96 = -1 mod p, 2^192 = 1 mod p 활용
if exp < 96 {
*self * Self::POWERS_OF_TWO[exp as usize]
} else if exp < 192 {
-*self * Self::POWERS_OF_TWO[(exp - 96) as usize]
} else {
self.mul_2exp_u64(exp % 192)
}
}
```

효율적인 축약:

```
fn reduce128(x: u128) -> Goldilocks {
let (x_lo, x_hi) = split(x);
let x_hi_hi = x_hi >> 32;
let x_hi_lo = x_hi & Goldilocks::NEG_ORDER;
let (mut t0, borrow) = x_lo.overflowing_sub(x_hi_hi);
if borrow {
```

```
t0 -= Goldilocks::NEG_ORDER;
}
let t1 = x_hi_lo * Goldilocks::NEG_ORDER;
let t2 = unsafe { add_no_canonicalize_trashing_input(t0, t1) };
Goldilocks::new(t2)
}
```

내적 연산 최적화

Goldilocks는 확장 필드 산술에 중요한 내적 연산을 특별히 최적화합니다:

```
fn dot_product<const N: usize>(lhs: &[Self; N], rhs: &[Self; N]) -> Self {
match N {
2 => {
// 2차 확장 필드에 특화된 최적화
let long_prod_0 = (lhs[0].value as u128) * (rhs[0].value as u128);
let long_prod_1 = (lhs[1].value as u128) * (rhs[1].value as u128);
let (sum, over) = long_prod_0.overflowing_add(long_prod_1);
let sum_corr = sum.wrapping_sub(OFFSET);
if over {
reduce128(sum_corr)
} else {
reduce128(sum)
}
```

```
}
_ => {
// 일반적인 경우의 최적화된 구현
// ...
}
}
}
```

SIMD 최적화 구현

AVX2 최적화

모든 31비트 필드는 AVX2를 통한 8-way 병렬 처리를 지원합니다:

```rust
pub struct PackedMersenne31AVX2(pub [Mersenne31; 8]);

impl Add for PackedMersenne31AVX2 {
fn add(self, rhs: Self) -> Self {
let lhs = self.to_vector();
let rhs = rhs.to_vector();
let res = mm256_mod_add(lhs, rhs, P);
unsafe { Self::from_vector(res) }
}
```

```
}
```

AVX2 모듈러 덧셈:

```
#[inline]
pub fn mm256_mod_add(lhs: __m256i, rhs: __m256i, p: __m256i) -> __m256i {
let sum = unsafe { _mm256_add_epi32(lhs, rhs) };
let sum_sub_p = unsafe { _mm256_sub_epi32(sum, p) };
let mask = unsafe { _mm256_cmpgt_epi32(sum, p) };
unsafe { _mm256_blendv_epi8(sum, sum_sub_p, mask) }
}
```

AVX-512 최적화

AVX-512가 지원되는 환경에서는 16-way 병렬 처리가 가능합니다:

```
#[cfg(all(
feature = "nightly-features",
target_arch = "x86_64",
target_feature = "avx512f"
))]
pub struct PackedMersenne31AVX512(pub [Mersenne31; 16]);
```

NEON 최적화 (ARM)

ARM 아키텍처에서는 NEON을 통한 4-way 병렬 처리를 지원합니다:

```
#[cfg(all(target_arch = "aarch64", target_feature = "neon"))]

pub struct PackedMersenne31Neon(pub [Mersenne31; 4]);
```

성능 비교 및 벤치마크

기본 연산 성능

| 연산 | Mersenne31 | BabyBear | KoalaBear | Goldilocks |

|------|----------|----------|----------|----------|

| 덧셈 | 매우 빠름 | 빠름 | 빠름 | 빠름 |

| 곱셈 | 빠름 | 매우 빠름 | 매우 빠름 | 매우 빠름 |

| 역원 | 보통 | 빠름 | 빠름 | 빠름 |

| FFT | 제한적 | 최적 | 좋음 | 최적 |

SIMD 성능 향상

- **AVX2**: 약 4-6배 성능 향상 (8-way 병렬화)
- **AVX-512**: 약 8-12배 성능 향상 (16-way 병렬화)
- **NEON**: 약 2-3배 성능 향상 (4-way 병렬화)

메모리 사용량

| 필드 | 단일 원소 | 패킹된 형태 (AVX2) | 압축률 |

|------|----------|----------|--------|

| Mersenne31 | 4 bytes | 32 bytes (8개) | 100% |

| BabyBear | 4 bytes | 32 bytes (8개) | 100% |

| KoalaBear | 4 bytes | 32 bytes (8개) | 100% |

| Goldilocks | 8 bytes | 32 bytes (4개) | 100% |

필드 선택 가이드

사용 사례별 권장사항

FFT 집약적 연산:

- **1순위**: BabyBear (two-adicity 27)
- **2순위**: Goldilocks (two-adicity 32, 하지만 64비트)
- **3순위**: KoalaBear (two-adicity 24)

일반적인 다항식 연산:

- **1순위**: Goldilocks (최고 성능)
- **2순위**: BabyBear (좋은 균형)
- **3순위**: Mersenne31 (단순함)

메모리 제약 환경:

- **1순위**: Mersenne31 (가장 단순)
- **2순위**: BabyBear/KoalaBear (31비트)
- **3순위**: Goldilocks (64비트)

특수 요구사항:

- **3차 거듭제곱근**: KoalaBear
- **높은 two-adicity**: Goldilocks
- **단순한 구현**: Mersenne31

확장 필드와 이항 확장

이항 확장 필드 구조

모든 필드는 $x^2 - w = 0$ 형태의 기약 다항식을 사용한 이항 확장을 지원합니다:

```
pub trait BinomialExtensionData<const D: usize> {
const W: Self::BaseField; // 기약 다항식의 계수
const DTH_ROOT: Self::BaseField; // D차 단위근
const EXT_GENERATOR: [Self::BaseField; D]; // 확장 필드 생성원
}
```

확장 필드 산술

곱셈 최적화:

```
// BabyBear⁴에서 w = 11인 경우
fn mul_w<A: Algebra<BabyBear>>(a: A) -> A {
// w * a를 직접 곱셈 대신 최적화된 연산으로 구현
specialized_mul_by_11(a)
}
```

보안 고려사항

상수 시간 구현

모든 필드 구현은 사이드 채널 공격을 방지하기 위해 상수 시간 연산을 보장합니다:

1. **분기 없는 조건부 연산**: 마스크를 사용한 조건부 선택

2. **균일한 메모리 접근**: 테이블 룩업 시 일정한 패턴

3. **타이밍 공격 방지**: 입력에 무관한 실행 시간

난수성 요구사항

```
impl Distribution<Mersenne31> for StandardUniform {
fn  sample<R: Rng + ?Sized>(&self, rng: &mut R) -> Mersenne31 {
loop {
let  next_u31 = rng.next_u32() >> 1;
let  is_canonical = next_u31 != Mersenne31::ORDER_U32;
if  is_canonical {
return Mersenne31::new(next_u31);
}
}
}
}
```

결론

Plonky3의 유한체 구현은 각각 고유한 수학적 특성과 최적화 기법을 가지고 있습니다:

- **Mersenne31**: 단순함과 기본 성능에 중점
- **BabyBear**: FFT와 Montgomery 산술의 균형
- **KoalaBear**: 특수 자기동형사상 지원
- **Goldilocks**: 최고 성능과 큰 필드 크기

SIMD 최적화를 통해 모든 필드가 현대 하드웨어에서 높은 성능을 달성하며, 상수 시간 구현으로 보안성을 보장합니다. 사용 사례에 따라 적절한 필드를 선택하는 것이 전체 시스템 성능에 중요한 영향을 미칩니다.